

CLAUDE.md — EmbodiedMAE-4M repository guide and experiment log

2026-09-28, yongyun branch

Contents

CLAUDE.md	1
What this is	1
Repo layout	2
Experiment programme — what is done and what is left	2
Two gaps that block the headline claim	3
On E6 and E7 being run elsewhere	3
E6 / E7 on the yongyun branch	3
The 600-epoch runs (2026-09-26) — these are the results	3
The 30 000-step sweep (screening only)	5
Environment	9
Common commands	9
Train 4M (the main entry point)	9
Train 3M	9
Cross-modal distillation	9
Validate / dump reconstructions	9
Sanity-check a dataset folder	9
Smoke-test the model definitions	10
Configuration model	10
Architecture (the part you'd otherwise have to read 4 files to learn)	11
Forward pass shape	11
Losses (forward_loss)	11
Param normalisation (4M only)	11
Dataset layout	12
Porting to another machine	12
Outputs	12
Things that look like dead code but aren't	13

CLAUDE.md

This file provides guidance to Claude Code (claude.ai/code) when working with code in this repository.

What this is

PyTorch implementation of **EmbodiedMAE** — a multi-modal Masked Autoencoder pre-trained on synthetic Sorghum plant data. Two model variants share the same encoder/decoder skeleton:

- **3M (embodied_mae.py)** — RGB + Depth + Point Cloud
- **4M (embodied_mae_4m.py)** — adds a parametric “spline/text” modality describing the plant’s procedural generation parameters

Both use the same Dirichlet-allocation token masking, transformer encoder, and per-modality decoder heads. The 4M variant is the active line of work; 3M is kept as the baseline it was derived from.

Repo layout

Configs and batch scripts were consolidated out of the repo root. **There are no `config.yaml` / `config_4m.yaml` at the root any more** — everything lives in:

- `configs/` — every YAML. `config.yaml` (3M), `config_4m.yaml` (4M), `config_4m_pretrain15k*.yaml` (pre-train), `config_4m_distill*.yaml` (cross-modal distillation), `config_e2*.yaml` (the E2 modality ablation), `config_e3*.yaml` (data scaling) and `config_e4*.yaml` (model scaling).
- `slurm/` — every `.sbatch` / launcher script. `e2_arm*.sbatch` launch E2 arms; `scale_arm_blackwell.sbatch` launches any E3 or E4 arm by slug (`sbatch --job-name=e3_1k slurm/scale_arm_blackwell.sbatch e3_1k`), deriving `configs/config_<slug>.yaml` and `outputs/<slug>/`.
- `data_split/` — the train/val/test split tooling (`make_split.py`, `move_split.py`, `reshuffle.py`).
- `eval/` — evaluation and analysis (`eval/analyze_e2.py`, `eval/eval_views_one_plant.py`, `eval/vis_pc_unpredicted.py`, `eval/eval_warmstart.py`, ...).
- `figures/` — anything that renders a figure, report or deck (`figures/make_results_pptx.py`, `plot*.py`, `gen*.py`, `regen*.py`).
- `export/` — dumps and artefact builders (`export/dump_param_examples.py`, `export_pc*.py`, `build*.py`).
- `sweeps/` — masking and visualisation sweeps.
- `tools/` — one-off data-prep and diagnostic scripts.
- **The repo root holds only what other code imports:** the two models, the two datasets, the training entry points, `validate*.py` and `utils.py`. That is the rule — if a file at the root is imported by nothing, it belongs in one of the folders above.

Scripts in those folders put the repo root on `sys.path` themselves (a four-line shim at the top), so `python eval/analyze_e2.py` works from the repo root without `PYTHONPATH` set. Keep the shim when adding a script there.

If a command or script still refers to a root-level `config_4m.yaml`, it is stale — the path is `configs/config_4m.yaml`.

Experiment programme — what is done and what is left

The programme is the E1–E10 matrix in the CVPR 2027 plan (the Google Doc is the source of truth for scope; this section is the source of truth for *state*). Paper deadline **Nov 13 2026**, internal results freeze **Oct 24 2026**.

Status as of 2026-09-20. Run state goes stale fast — the table records which experiments have been *built*, which is durable. For live progress use `squeue -u $USER`, `outputs/<run>/training_history.json`, and `outputs/<run>/config.json` (which records what a run actually used).

	experiment	state	runs
E1	headline pretrain	done	<code>4m_pretrain_15k_v2_depthfix_qal</code> , 1000/1000 ep, global batch 256
E2	modality value-add	done	all four arms 600/600; results in <code>reports/RESULTS_DECK_2026-09-20.md</code>
E3	data scaling	2 of 3	<code>e3_3k</code> yes <code>e3_10k</code> yes; <code>e3_1k</code> resuming from epoch 5654/6169
E4	model scaling	1 of 2	<code>e4_small</code> yes; <code>e4_large</code> resuming from epoch 250/600
E5	view regime	not built	—
E6	masking / noise	600-epoch pass done; 1 000-epoch continuation pending (yongyun branch)	structured masking trades clean PC accuracy for robustness: worse clean F1, near-zero degradation under corruption — see E6 / E7 on the yongyun branch
E7	loss study	first pass done (yongyun branch)	QAL > Chamfer > Sinkhorn-only on F1; Sinkhorn trades precision for recall; <code>model.loss_name</code> now also takes sinkhorn — see below
E8	baselines	not built	—
E9	latent analysis	not built	—
E10	real-data OOD	partial	<code>OOD_EVAL_rgb2pc.md</code> and the <code>eval_rgb2pc*.py</code> scripts

On the two resuming arms. `e3_1k` and `e4_large` both died at exactly 2026-09-19T15:51:27 on nova26-gpu-2 — same second, same node, no traceback, host RSS far under request. That is a node-level event on the preemptible scavenger partition, not a fault in either run, and the launcher’s auto-resume bounds the loss at `save_freq` epochs.

e3_1k's `training_history.json` was left truncated mid-write; its val series is recoverable from the run log in `logs/`.

On waiting for GPUs. When these jobs sit PENDING for days, check whether it is your request before shrinking it: a 1-GPU / 1-CPU / 1 GB / 10-minute test job placed at the *same* time as the full 2-GPU / 320 GB request, which means the nodes are held by a reservation and no amount of trimming helps.

Two gaps that block the headline claim

1. There is no downstream linear probe, and the plan says that is the metric. Locked decision 6.4 makes the value-add metric a linear probe on height, leaf angle, leaf count and biomass — explicitly *not* reconstruction loss. `eval/analyze_e2.py` compares arms on `val_pc_chamfer`, which is the correct arm-invariant *monitoring* signal and is not what 6.4 asks for. So E2, E3 and E4 will all finish and produce chamfer curves with no probe number attached.

Three of the four targets are already in `features.csv` at the split root (15 000 rows keyed by `plant`, which joins to `Sorghum_<plant>_<view>`): `stem_length` → height, `n_leaves` → leaf count, and leaf angle is either `roll_mean` (twist) or `branch_mean` (insertion) — the spline YAMLS carry `roll_angle` and `branching_angle` as distinct fields and the plan does not say which it means. **Biomass is in neither `features.csv` nor the spline params** and has to be derived (cheapest proxy: `n_leaves × leaf_len_mean`).

Build and validate this against an existing checkpoint *before* the arms finish — if the probe is broken or these targets carry no linear signal, that is much cheaper to learn now than after the runs are unrepeatable.

2. No baselines exist (E8), and the plan ranks that the #1 reject risk. Nothing in the repo addresses it, and every baseline is itself a training run that has to fit before the Oct 24 freeze, so the *decision* about what to compare against is more time-critical than the runs.

On E6 and E7 being run elsewhere

Masking/noise and the loss study are a collaborator's, so nothing in this repo should schedule them. Their results still have to land in the same table as everything above, which means the comparison only holds if they match this programme on the three things that silently break it: the **same** 70/15/15 split of `Sorghum_15K` (seed 42), the **same** global batch of 32, and a budget stated in **optimizer steps** rather than epochs. Confirm those three before their numbers are merged, not after — `outputs/<run>/config.json` records all three for any run that used this codebase.

That leaves **E5, E8, E9 and E10 plus the linear probe** as the work owned here.

E6 / E7 on the yongyun branch

Results from the collaborator side, 2026-09-21. Every run below is **4M base, recipe-B occlusion + structured masking, 3 000 plants, one view per plant per epoch, 30 000 optimizer steps** (94 steps/epoch → 320 epochs; best checkpoints at epochs 300–320, i.e. still improving when the cosine schedule ends), evaluated on a fixed 225-view clean val set and, for the occluded columns, under one reference corruption shared by all runs (`occlusion.eval_occlusion, 1× noise`).

The 600-epoch runs (2026-09-26) — these are the results

Twelve runs, `train_4m_paper.sbatch + config_4m_paper_*.yaml`: **56 400 optimizer steps = 600 epochs** at 94 steps/epoch (3 000 plants, one view per plant per epoch), one cosine from scratch with 19 warm-up epochs, global batch 32 (8 per GPU × 4 L40S), fixed 225-view clean val. Arms that carry a claim run at **seeds 1 and 2**, so every gap can be read against the run-to-run spread.

600 epochs is enough here: PC F1@0.03 climbs from 0.60 to 0.73 between epochs 200 and 250 and is flat over the last four validation points (e.g. 0.733, 0.733, 0.735, 0.734), so arms are compared after the transition, not before it. It is, however, **a short schedule next to the programme's 197 400 steps** and has to be labelled that way wherever these rows are reported. An earlier attempt at the full 197 400-step budget was stopped at ~580 epochs and is kept in `outputs/_aborted_2100budget/`; its numbers are not comparable because its cosine was still at 85 % of peak LR there.

E6 — structured masking: worse clean reconstruction, better robustness

arm	token masking	F1@0.03 s1 / s2	recall@0.03 s1 / s2	F1@0.01 s1 / s2	PC Chamfer s1 / s2	Depth MSE s1 / s2
paper_maskoff	off (uniform)	0.734 / 0.842	0.650 / 0.773	0.230 / 0.352	0.00259 / 0.00181	0.0426 / 0.0470
paper_maskedge	edge weighting only (length_scale 0.02)	0.611 / 0.655	0.485 / 0.541	0.184 / 0.206	0.00422 / 0.00358	0.0380 / 0.0383
paper_maskblob	blobs only (center_bias 0)	0.613 / 0.673	0.487 / 0.571	0.180 / 0.197	0.00421 / 0.00332	0.0405 / 0.0394
paper_maskboth	blobs + edge (the old default)	0.608 / 0.602	0.480 / 0.475	0.176 / 0.175	0.00435 / 0.00436	0.0390 / 0.0385

Both **maskoff** seeds beat all six structured seeds on every PC metric, and the gap (0.15–0.23 F1@0.03) is far larger than the seed spread of any structured arm (0.005–0.06). Turning structured masking off raises recall@0.03 from ~0.48 to ~0.71, which is the same quantity the radial analysis below blamed on missing leaf tips — edge-biased masking spends the mask budget on the frame border and starves exactly the outer plant. Blobs alone, edge bias alone and both together are indistinguishable from each other.

Robustness is the other half. The same runs, scored under the occluded validation pass — one fixed reference corruption (procedural leaves at 9–16 per image plus 1× sensor noise, eval seed 42), identical for every run:

arm	PC Chamfer clean s1 / s2	PC Chamfer occluded s1 / s2	degradation (occ / clean) s1 / s2
paper_maskoff	0.00259 / 0.00181	0.00285 / 0.00199	1.10 / 1.10
paper_maskedge	0.00422 / 0.00358	0.00432 / 0.00373	1.02 / 1.04
paper_maskblob	0.00421 / 0.00332	0.00433 / 0.00348	1.03 / 1.05
paper_maskboth	0.00435 / 0.00436	0.00444 / 0.00438	1.02 / 1.00
paper_paramsv2	0.00420 / 0.00435	0.00426 / 0.00435	1.01 / 1.00

Structured masking does what it was built for: corruption barely moves its error (0–5 %), while uniform masking loses 10 %. At this corruption strength, though, the robustness does not pay for itself — **maskoff** under corruption is still better than every structured arm on *clean* input. So the finding is a trade-off, not a win either way, and it depends on how hard the test corruption is. Whether the curves cross at stronger corruption is the open question (`eval_robustness.py`, below). The occluded pass logs Chamfer, RGB and depth only — F1/recall under corruption are not recorded yet.

What structured masking already wins on, measured:

property	structured (10 runs: mask arms, loss arms, params v2)	maskoff (2 runs)
PC Chamfer degradation under the reference corruption	1.00–1.05×	1.10×
Depth MSE (clean)	0.0363–0.0407 — every one of the 10 runs	0.0426 / 0.0470
F1@0.03 seed-to-seed spread	0.006 (maskboth)	0.108

The depth result has no exceptions: all ten structured runs beat both **maskoff** seeds. The stability gap is large but ambiguous — a model can be reproducible because it settles on a lower plateau. These are supporting evidence; the headline claim needs one of the tests in “Showing where structured masking helps” below.

Caveat before this is quoted: **maskoff**’s two seeds are 0.734 vs 0.842, a much wider spread than any other arm, so the direction is solid but the magnitude is not. A third **maskoff** seed is the cheapest way to pin it (~10 h). Note also that depth goes the other way: structured arms reach 0.038–0.040 Depth MSE against 0.043–0.047 for **maskoff**.

Spline params: v2 + pose/scale helps once its loss weight is fixed Same masking (**maskboth**), same seeds; v1 is the **paper_maskboth** pair. `spline_loss_weight` is 5 for v1 and **0.6** for v2, which compensates for v2’s z-scored targets being ~6× larger — the confound that made the earlier 30k comparison read backwards.

run	F1@0.03 s1 / s2	F1@0.01 s1 / s2	recall@0.03 s1 / s2	PC Chamfer s1 / s2
v1 (paper_maskboth)	0.608 / 0.602	0.176 / 0.175	0.480 / 0.475	0.00435 / 0.00436
v2 + pose/scale (paper_paramsv2)	0.619 / 0.615	0.224 / 0.215	0.483 / 0.483	0.00420 / 0.00435

F1@0.01 improves by ~25 % (0.175 → 0.219) against a seed spread of 0.001–0.009, so this one is real. The gain is concentrated at the tight threshold, i.e. in placing points precisely, which is what camera pose plus the normalisation radius should buy.

E7 — point-cloud loss: QAL leads, Sinkhorn trades precision for recall One seed each, masking fixed at `maskboth`; the QAL row is `paper_maskboth_s1`.

loss	F1@0.03	recall@0.03	precision@0.03	F1@0.01	PC Chamfer	Depth MSE
QAL (t=0.01, α=100)	0.608	0.480	0.850	0.176	0.00435	0.0390
Chamfer (pc_loss_weight 10)	0.578	0.450	0.846	0.144	0.00435	0.0363
Sinkhorn only (blur 0.02, 2 048 pts)	0.474	0.710	0.359	0.136	0.01032	0.0407

Sinkhorn is the interesting one: it is the only loss that spreads predictions over the whole plant (recall 0.71 vs 0.45–0.48) but it places them badly (precision 0.36 vs 0.85). QAL and Chamfer do the opposite. The combination (QAL with a Sinkhorn auxiliary term, which `sinkhorn_loss_weight` already supports) is the obvious next arm and has not been run.

The 30 000-step sweep (screening only)

Everything below this line is the earlier 30 000-step pass. It sits *before* the F1 transition and its gaps were inside the seed gap, so it is kept as screening evidence — which settings to drop — and not as a result.

Held fixed in every run — including the edge weighting. Token masking: `model.structured_mask` with `prob: 0.5` (half the batches; the rest use the uniform shuffle), `center_bias: 1.5` (a smooth random field biased toward the frame border, so masked blobs gather at the edges and the centre plant stays visible) and one field shared across RGB, depth and the PC; train-only, so validation always uses the uniform mask. Input occlusion: procedural leaves entering from outside the frame (`reach: [0.6, 1.2]`, 9–16 leaves, on every training sample), interleaved in depth (`depth_quantile: [0.0, 0.8]`), plus sensor noise. **No run in this screening pass varies `center_bias`, `prob` or `reach`** — that question is answered by the 600-epoch `paper_mask*` arms above, and the answer is that structured masking costs PC reconstruction.

Against the three merge conditions above. Split: same `Sorghum_15K` root yes. Global batch: 8×4 GPUs = 32 yes. Budget in steps: 30 000 yes — but that is a *selection* schedule, ~15 % of the programme’s 197 400 (`e3_3k`, same 94 steps/epoch, runs 2 100 epochs), so only the ranking transfers, not the absolute numbers. The 3 000 plants are drawn by `training.num_train_plants` with seed 42; that is **not** the same draw as main’s `data.max_plants / plant_subset_seed`, so it is not `e3_3k`’s 3k.

Noise floor — read every table against this. Training had no seed until `training.seed` was added here, so `recipe_b` (unseeded) and `params_v1_seed1` (seed 1) are the *same config* run twice. Their gap is the only run-to-run error bar in this section:

	val loss	val(occ) loss	PC Chamfer	F1@0.01	F1@0.03	EMD	Depth MSE
seed-to-seed gap	0.027	0.014	0.6 %	1.7 %	0.3 %	1.4 %	11 %

Which F1 threshold. F1@0.03 is the headline column: its ceiling is ~1, so the value reads directly as a fraction of a perfect reconstruction. F1@0.01 is kept beside it. Its ceiling is only 0.871 — two independent 8 196-point draws of the *same* ground-truth cloud score that against each other (0.995 at 0.02, 0.999 at 0.03) — so 0.154 is ~18 % of reachable, not 15 %. It is also slightly the more discriminating of the two here (recipe spread / seed gap $3.1\times$ vs $2.7\times$), and it is the column that exposes the leaf-tip failure below. Do not move to a threshold where $F1 \approx 0.9$: every arm saturates there and the gaps compress.

E6 — structured-mask blob size × sensor noise

run	blob size (length_scale)	train noise (rgb/depth/pc)	val loss	val(occ) loss	PC Chamfer	F1@0.03	F1@0.01	RGB MSE	Depth MSE
recipe_a_ls 035_noise1x	0.35	1× (0.03 / 0.015 / 0.008)	0.5829	0.5964	0.004703	0.5832	0.1582	0.6738	0.03350
recipe_b_ls 020_noise1x	0.20	1×	0.5623	0.5853	0.004644	0.5856	0.1567	0.6649	0.03355
recipe_c_ls 020_noise0p5x	0.20	0.5×	0.5908	0.5993	0.004716	0.5816	0.1536	0.6715	0.03219
recipe_d_ls 020_noise2x	0.20	2×	0.5891	0.6010	0.004680	0.5817	0.1503	0.6610	0.03221
params_v1_seed1 (= B, seed 1)	0.20	1×	0.5891	0.5996	0.004670	0.5841	0.1541	0.6773	0.02997

Nothing is bold: no gap in this table exceeds the seed-to-seed gap.

B has the best clean and occluded val loss, but **B run a second time lands behind A and level with C and D**: every recipe gap (A-B 0.021, C-B 0.029, D-B 0.027 val loss) is the size of the seed gap. The sweep does not rank these recipes — and the order of A and B flips with the F1 threshold (A > B at 0.01, B > A at 0.02 and 0.03). The default stays B (length_scale 0.20, 1× noise) — best observed, not shown better. Separating them needs ≥ 3 seeds per arm.

Where the model fails: leaf tips, not the frame edge. Trained recipe-B model, 12 val plants, 4 mask draws; radius 0 = image centre, 1 = corner (for the PC, the xy radius of the unit-normalised cloud). Errors on *masked* patches:

radius	0–0.2	0.2–0.4	0.4–0.6	0.6–0.8	0.8–1
plant share of patches	71 %	40 %	16 %	3 %	0.2 %
P(masked), uniform mask	0.64	0.61	0.60	0.60	0.62
P(masked), structured (center_bias 1.5)	0.64	0.70	0.81	0.86	0.89
RGB MSE, all masked patches	0.18	0.17	0.09	0.02	0.002
RGB MSE, plant patches only	0.20	0.25	0.29	0.27	0.60
PC: target point → nearest prediction	0.025	0.042	0.072	0.119	0.233

The edge bias does what it says — masking rises from 0.64 to 0.89 toward the corner — but the corner is background, which is why the error there is near zero. On the plant itself error *grows* outward, and PC error on the outermost points is $\sim 10\times$ the centre’s: the model misses leaf tips. That is the low F1@0.01 (precision 0.40, recall 0.11 at the end of training — predicted points sit on the dense core and do not reach the tips). So much of the edge-biased mask budget is spent on background; masking the plant’s *outer parts* would target the actual failure. The 600-epoch arms above confirm the mechanism from the other side: switching structured masking off raises recall@0.03 from ~ 0.48 to ~ 0.71 . Preview of the masks: [reports/mask_preview_recipe_b.png](#), [reports/mask_blob_size.png](#) (uniform vs blob sizes at a fixed mask count) and [reports/noise_levels.png](#) (the $0.5\times / 1\times / 2\times$ sensor noise).

Spline params: do they help, and does fixing their encoding help more? Measured on the data before changing anything: 5 of the 9 plant dims (panicle_*) are constant; the per-sample PC normalisation removes absolute size (corr(stem_length, cloud height) is 0.68 raw, 0.08 after normalisation); and the params are in the plant’s world frame while the target is in the camera frame (*_nc_cam.ply is exactly worldToCamera @ world), with the camera_pose.json that links them never read. model.param_encoding: v2 (z-scored, sin/cos angles, constants dropped, leaf count added) and model.geometry_cond: true (camera rotation + normalisation radius as an always-visible token) address those. Same seed, only the params path differs:

run	PC Chamfer	F1@0.01	F1@0.02	F1@0.03	EMD	RGB MSE	Depth MSE
params_v1_seed1	0.004670	0.1541	0.4097	0.5841	0.2966	0.6773	0.02997
params_v2cond_seed1	0.004967	0.1587	0.4057	0.5748	0.2948	0.6758	0.03256

Bold = better by more than the seed-to-seed gap; every other gap is within noise.

`eval_param_oracle.py` then asks each model directly: all param tokens visible, the sample’s own params vs the val-set mean; identical RGB/depth/PC masks in both passes. Positive = worse without the sample’s own params:

checkpoint	PC Chamfer	F1@0.01	F1@0.02	RGB MSE	Depth MSE	occluded PC Chamfer
v1	+7.0 %	+1.4 %	+2.1 %	-1.7 %	+7.1 %	+7.9 %
v2 + pose/scale	+4.6 %	+5.3 %	+2.5 %	+0.4 %	-0.2 %	+4.5 %

The params already help in v1 (7 % Chamfer, well above the 0.6 % noise floor). v2 + pose/scale looked 6 % worse here, and the suspected cause — v2’s z-scored Smooth-L1 targets being $\sim 6\times$ larger at the same `spline_loss_weight`: 5, so the param term steals gradient from the PC term — turned out to be right: **at `spline_loss_weight`: 0.6 and 600 epochs v2 wins** (F1@0.01 0.219 vs 0.175), see the paper-run section above. The oracle numbers below still stand; they are measured within a single checkpoint.

E7 — point-cloud loss study (running) Loss magnitudes measured on the trained recipe-B model set the weights: QAL 0.060, Chamfer 0.0053 (11 $\times$ smaller), Sinkhorn ~ 0.055 . Chamfer therefore needs `pc_loss_weight` ≈ 10 to pull as hard as QAL at 1. `loss_name: sinkhorn` means Sinkhorn *alone* (Sinkhorn was only an auxiliary term before), on 2 048 sampled points.

loss	arms	runs
QAL	t = 0.005 / 0.01 / 0.02 at $\alpha = 100$; $\alpha = 30$ / 300 at t = 0.01	<code>loss_q_t005</code> , <code>params_v1_seed1</code> (t=0.01, $\alpha=100$), <code>loss_q_t020</code> , <code>loss_q_a030</code> , <code>loss_q_a300</code>
Chamfer	<code>pc_loss_weight</code> 3 / 10 / 30	<code>loss_c_w03</code> , <code>loss_c_w10</code> , <code>loss_c_w30</code>
Sinkhorn only	blur 0.01 / 0.02 / 0.05 at weight 1; blur 0.02 at weight 3	<code>loss_s_b010_w1</code> , <code>loss_s_b020_w1</code> , <code>loss_s_b050_w1</code> , <code>loss_s_b020_w3</code>

Job array 15884575 (`train_4m_loss.sbatch`); the Sinkhorn-only path passed its smoke test first. The array allows two arms at a time, but with the mech-ai account at its 17-GPU cap it has run one at a time (~ 5 h per arm). Read it with `python compare_recipes.py --loss-study`, which shows only metrics computed identically for every arm — val loss is a different quantity in each.

Finished so far (2026-09-22): 1 of 11 new arms.

run	loss	setting	PC Chamfer	F1@0.01	F1@0.02	F1@0.03	EMD	RGB MSE	Depth MSE
<code>params_v1_seed1</code>	QAL	t=0.01, $\alpha=100$	0.004670	0.1541	0.4097	0.5841	0.2966	0.6773	0.02997
<code>loss_q_t005</code>	QAL	t=0.005, $\alpha=100$	0.004700	0.1540	0.4060	0.5822	0.2970	0.6910	0.02898

Halving the QAL threshold moves nothing past the noise floor.

What this branch adds, and what will conflict with main

- Config keys: `training.num_train_plants`, `training.views_per_epoch`, `training.max_steps`, `training.seed`, `occlusion.eval_occlusion`, `model.param_encoding`, `model.geometry_cond`, and `loss_name: sinkhorn`. All default off; with none set, a model is bit-identical to before (same parameters, init and loss).
- Main implements plant subsetting and view sampling separately (`data.max_plants`, `data.view_sampling`). Keep one when merging; they will conflict in `train_sorghum_4m.py` and the datasets.
- New files, at the root because this branch predates main’s reorganisation: `view_sampler.py`, `compare_recipes.py`, `eval_param_oracle.py`, `compute_param_stats.py`, `config_4m_{recipe,params,loss}*.yaml`, `train_4m_{recipe,params,loss}.sbatch`. On merge they belong in `configs/`, `slurm/` and `eval/`.
- Slurm: submitted under `--account=mech-ai`, whose GPU cap (17) is shared across the lab. Arrays run **two at a time = 8 GPUs**, the ceiling this project keeps.
- Launchers: `train_4m_paper.sbatch` (the twelve 600-epoch runs), `train_4m_recipe.sbatch` / `train_4m_loss.sbatch` (the 30k screening), `train_4m_continue.sbatch` (resume an existing run at a larger `--max_steps`; unused now that the runs are single-cosine).

Showing where structured masking helps — the three legitimate tests The claim the method needs is “structured masking makes the model better at the thing it is for”. Clean reconstruction is not that thing, and on it the answer is no. Three tests could show a real advantage; each is worth running, and if none shows one, the honest write-up is the trade-off above.

1. **Stronger corruption and test-time blob masks, on TEST** — `eval_robustness.py --split test` scores `best_model.pth` (chosen on val loss, so val numbers carry selection bias; report test) on one view per test plant (2 250 plants, view fixed by seed — 10× the 225-view val subset, no plant counted twice). Levels: `clean`, `1x`, `noise2x`, `noise4x`, `leaves2x`, `leaves2x_noise2x`, and two **blob-mask** levels, `blobmask` (clean input) and `blobmask_1x` (1× corruption), which impose the `maskboth` blob geometry on *every* model’s token mask at test time. Every other evaluation masks tokens uniformly — the regime `maskoff` trained on — so the blob levels are the first test on structured masking’s own terms: filling large contiguous holes, the shape real occlusion takes. Queued as two 1-GPU jobs (`.smoke/eval_robustness.sbatch`, `EVAL_RUNS / EVAL_OUT`, results in `reports/robustness_test_{A,B}.json`), after the 1 000-epoch continuation.
2. **The programme’s actual metric** — decision 6.4 scores value-add with a linear probe on height, leaf angle, leaf count and biomass, *not* reconstruction. Structured masking could help the representation even while it hurts reconstruction; the probe is the test (see “Two gaps”).
3. **Real data (E10)** — real occlusion is what the synthetic corruption imitates; an advantage there is the strongest possible evidence.

Operational notes (2026-09-28)

- **The `/work/mech-ai-scratch` filesystem is degraded, cluster-wide.** It is an NFS mount (`novastor010:/stor010/mech-ai-scratch`) at **99 % full — 150 of 152 TB**. On a compute node (`nova18-wide-10`), `import numpy` took 175 s and `import torch`, `import open3d` and listing 500 dataset folders each ran past 300 s; `1 s` takes 4.6 s in `/work/mech-ai-scratch/yongyun` and 12 s in the dataset root, against 4 ms in `/home` and 0.7 s in `/work/mech-ai`. This repo’s own checkpoints are ~0.6 TB (~0.4 % of the used space), so freeing them is housekeeping, not a fix; the fix is HPC support and the lab freeing space. Symptoms so far: continuation task `16613000_0` held 4 GPUs for 7 h without leaving the dataset scan; the first robustness eval hit its 4 h limit the same way. **Do not submit GPU jobs until time `python -c "import torch"` is back under a minute** — a stalled job holds 4 GPUs doing nothing.
- **Auto-submit watcher:** `.smoke/watch_and_submit.sh` (log in `.smoke/watch.log`) probes `import torch` every 10 min; after two healthy probes it pre-builds the index cache with `.smoke/build_index_cache.py` (torch-free, same keys as the loaders, val/test/train), then submits the twelve-run continuation to 1 000 epochs (8 GPUs) and the two test-split robustness evals after it.
- **Dataset folder index is now cached** in `.index_cache/` (per split, per 3M/4M). The first run pays one scan; every later run and evaluation reads the cache. `SORGHUM_INDEX_REFRESH=1` forces a rescan.
- **`train_4m_continue.sbatch` takes RUNS colon-separated.** `sbatch --export` splits its argument on commas, so a comma list silently delivered only the first run and array job `16613000` lost eleven of twelve tasks.
- **Continuation checkpoints every 100 epochs** (`SAVE_FREQ`, default 100 in `train_4m_continue.sbatch`): at 1.3 GB each, the configs’ 30-epoch cadence would add ~200 GB to a 99 %-full disk; 100 adds ~60 GB.
- **Pending:** all twelve `paper_*` runs resumed from epoch 600 to 1 000 (`TARGET_STEPS=94000`), then `eval_robustness.py --split test` on the results.

Checkpoint retention — what the paper needs Keep, per run: `best_model.pth` (the reported numbers come from it — it is not necessarily the last epoch), **the latest checkpoint** until the run is finished (it is the resume point), and `training_history.json` + `config.json` always (kilobytes; they back every curve and every “what did this run use” question). Intermediate checkpoints are not needed — the val curves in the history replace them. By group:

runs	keep	delete when space is needed
<code>paper_*</code> (the results)	best + latest (epoch 600 now, 1 000 after the continuation) + history	intermediates (19 per run, ~297 GB)
<code>params_v1_seed1</code> , <code>params_v2cond_seed1</code>	best, if the oracle table is used	intermediates
30k screening (<code>recipe_*</code> , <code>loss_*</code>)	history	all weights (~157 GB)
<code>_aborted_*</code>	history (supports “600 epochs is enough”)	all weights (~14 GB)

Two cautions: other project folders under `/work/mech-ai-scratch/yongyun` (e.g. `embodiedmae4m`, linked to the epoch-760 results) may hold models the paper cites — check before deleting there. And scratch is not backed up,

so copy the handful of `best_model.pth` files the paper reports off scratch (`/work/mech-ai` or external storage). Nothing has been deleted yet.

What to run next, in order

1. **A third maskoff seed** — its two seeds are 0.734 and 0.842 on F1@0.03, the widest spread of any arm, and it carries the headline claim (~10 h).
2. **QAL + Sinkhorn auxiliary** (`sinkhorn_loss_weight > 0` on top of `loss_name: qal_loss`): Sinkhorn alone reaches recall 0.71 at precision 0.36, QAL alone 0.48 at 0.85, so the combination is the obvious arm.
3. **v2 params + maskoff** — both wins are measured separately, never together.

Environment

Conda env name is `det` (Python 3.12, PyTorch 2.5 + CUDA 12.4; see `environment.yml`). On Nova it lives at `/work/mech-ai/alloy/.conda/envs/det`. Activate with `conda activate det` before running anything.

On Blackwell / `sm_120` GPUs (RTX PRO 6000) `det` will not work — those need the CUDA 12.8 build in the separate `det_cu128` env.

Common commands

All commands assume `cwd = repo root` and the env active.

Train 4M (the main entry point)

```
# Single GPU
python train_sorghum_4m.py --config configs/config_4m.yaml --world_size 1

# Multi-GPU via torchrun (preferred - sets LOCAL_RANK/RANK/WORLD_SIZE)
torchrun --standalone --nproc_per_node=4 train_sorghum_4m.py --config configs/config_4m.yaml

# Multi-GPU via mp.spawn fallback (set distributed.world_size in the YAML)
python train_sorghum_4m.py --config configs/config_4m.yaml
```

Train 3M

```
python train_sorghum.py --config configs/config.yaml # single GPU
python train_sorghum_multi.py --config configs/config.yaml # mp.spawn
```

Cross-modal distillation

`train_sorghum_4m_distill.py` trains the 4M model so that **any single modality, with the other three fully masked, reconstructs all four** — a frozen full-modal teacher supplies token-aligned decoder features and a CLS latent as the privileged target, and the student is warm-started from the same checkpoint.

```
python train_sorghum_4m_distill.py --config configs/config_4m_distill_15k_rgb2pc.yaml
```

Validate / dump reconstructions

```
python validate.py --checkpoint outputs/<run>/best_model.pth \
  --output_dir vis_val --config configs/config.yaml --num_samples 6
```

Sanity-check a dataset folder

```
python sorghum_dataset.py /path/to/Sorghum_15K # 3M
python sorghum_dataset_4m.py /path/to/Sorghum_15K # 4M (requires *_spline.yml)
```

Smoke-test the model definitions

```
python embodied_mae.py          # builds embodied_mae_base, one forward pass on dummy tensors
python embodied_mae_4m.py       # same for 4M
```

There is no test suite, no linter, and no Makefile.

Configuration model

Both training entry points layer config in this order: YAML → CLI flags → defaults. The YAML is the source of truth; CLI flags only override specific keys. Notable keys:

- `data.data_root` expects `<root>/train/`, `<root>/val/` and `<root>/test/` siblings, each containing one folder per sample (see “Dataset layout”).
- `data.view_sampling: true` — each plant contributes **one randomly chosen view per epoch**, so an epoch over 105 000 train samples is 10 500 items, not 105 000. `view_seed` must be identical across arms of an ablation so they see the same views in the same order.
- `data.max_plants` — caps the **train** split at a nested random subset of N plants (`plant_subset_seed` fixes the draw); `null` uses all of them. This is what E3 varies. The subsets nest, so $1k \subset 3k \subset 10k$ and a step of the curve is never partly a change of sample composition. It selects **plants, keeping all ten views** — dropping views would change the augmentation regime that decision 6.1 fixes, which is E5’s question, not E3’s. `train_sorghum_4m.py` passes it to the train dataset only; a capped val/test would score the arms on different yardsticks.
- `model.model_size` $\in \{\text{small, base, large, giant}\}$ (3M) or $\{\text{small, base, large}\}$ (4M). This is what E4 varies: 4M builds at 25.1 M / 114.3 M / 332.2 M params.
- `model.mask_ratio` is the **total** masking fraction across all modalities; the per-modality split is sampled from `Dirichlet( $\alpha$ =dirichlet_alpha)` once per batch.
- `model.active_modalities` — comma-separated subset of `pc,rgb,depth,text`. Restricting it is what the E2 ablation varies; everything else stays fixed.
- `model.loss_name` $\in \{\text{chamfer, qal_loss}\}$ with `qal_threshold` / `qal_alpha` / `qal_use_squared`. Current runs use `qal_loss`.
- `model.pc_loss_weight` scales the PC term. Chamfer values are tiny relative to MSE, so this is typically $O(10)$ when Chamfer is selected.
- `model.spline_loss_weight` (4M only) scales the Smooth-L1 param-regression loss.
- `model.depth_norm_type` $\in \{\text{minmax, standard}\}$ — applied to the **target** before depth MSE; the model learns to predict normalised values directly.
- `checkpointing.resume`: path to a `.pth` to resume from, or `null` for scratch.
- `distributed.world_size > 1` triggers DDP. `train_sorghum_4m.py` also accepts being launched under `torchrun`, in which case `LOCAL_RANK` is honoured and `world_size` is inferred from `env`.

Global batch is `batch_size` × `world_size`. `batch_size` in the YAML is per-GPU, so an ablation must adjust it to the GPU count to keep the global batch constant — a global-batch difference between arms confounds the comparison. The E2 launchers use 16×2 on the 2-GPU Blackwell nodes and 8×4 on the 4-GPU A100 nodes, both reaching 32.

Size an ablation in optimizer steps, not epochs — and do not “fix” epoch counts that disagree. Under `view_sampling` an epoch is one view per plant, so *epoch size is the plant count*: at global batch 32 the full train split gives 329 steps/epoch, but a 1 000-plant subset gives 32. Arms that differ in data scale therefore need very different epoch counts to receive the same number of gradient updates, and the E3 configs look wrong at a glance because of it:

arm	plants / model	steps/ep	epochs	total steps
e3_1k	1 000	32	6 169	197 408
e3_3k	3 000	94	2 100	197 400
e3_10k	10 000	313	631	197 503
e4_small / e4_large	25.1 M / 332.2 M	329	600	197 400
e2_pcrgrbdt	10 500, base	329	600	197 400

Equalising those epoch counts would hand the 10k arm ten times the updates of the 1k arm, and the resulting curve would measure data scale and compute jointly with no way to separate them afterwards. `warmup_epochs`, `val_fre`

`q` and `save_freq` are scaled by the same per-arm factor, so every arm warms over the same fraction of its schedule and lands ~24 val points.

That shared 197 400-step budget is `e2_pcrgrbdt`'s, which is why **`e2_pcrgrbdt` is also E3's full-data point and E4's base-model point** rather than a separate run — and why the E3/E4 launcher's resources are deliberately byte-identical to the E2 one. `e3_10k` (10 000 plants) and `e2_pcrgrbdt` (10 500) are within 5 % of each other, so that pair doubles as the only run-to-run error bar in the experiment matrix.

`persistent_workers` is deliberately **off** in the dataloaders and must stay off: persistent workers hold a copy of the dataset made at first iteration, so `train_ds.set_epoch(epoch)` would never reach them and view sampling would silently freeze at epoch 0. The cost is that workers respawn every epoch, which matters most for `e3_1k`, whose epoch is only ~1 000 items.

Architecture (the part you'd otherwise have to read 4 files to learn)

Forward pass shape

1. **Embed each modality** to (B, L_m, D):
 - RGB / Depth → `PatchEmbed` (Conv2d patchification → 196 tokens for 224×224, patch=16).
 - Point cloud → `PointCloudEmbed`: FPS samples `num_pc_tokens`=196 centres, kNN groups `group_size`=3 2 neighbours per centre, two PointNet-style Conv1d blocks produce per-token features.
 - (4M only) Spline params → `TextLeafEmbed`: char embeddings + learned positional → mean-pool → linear → D. `n_text_tokens` = 1 + `max_leaves` (1 plant token + up to 24 leaf tokens).
2. **Add positional + modality embeddings** (each modality has its own learned modality bias).
3. **Dirichlet masking** in `random_masking_dirichlet`: a single Dirichlet draw decides the visible-token split across modalities for the whole batch step. Each sample's visible indices are independently random, but the per-modality count is identical batch-wide — this avoids zero-token batch elements that would corrupt encoder norms. The 4M variant also enforces `min_mask_ratio`=0.25 per modality.
4. **Encoder**: visible tokens from all modalities are concatenated with a CLS token and fed through `depth` shared transformer blocks (`embed_dim`=768 for base).
5. **Decoder**: project to `decoder_embed_dim`=512, restore mask tokens at original positions per modality, add decoder positional embeddings, run `decoder_depth`=8 transformer blocks, then split back into modality-specific heads:
 - RGB / Depth → linear → `patch_size`² × C per token (unpatchified for visualisation).
 - PC → FoldingNet-style upsampling: each of 196 PC tokens generates `points_per_token` = `target_points` // `num_pc_tokens` 3D points by concatenating its 512-D feature with a 2D grid coordinate and passing through an MLP. `target_points` defaults to 10000, trimmed/padded to exactly that on output.
 - (4M only) Spline → 2-layer MLP → `N_PARAMS`=9 floats per token, range [0, 1].

Losses (forward_loss)

- **RGB**: per-patch MSE, optionally with `norm_pix_loss` (per-patch normalisation), masked mean.
- **Depth**: per-patch MSE on **normalised** target (per-image min-max or standard); the prediction is compared to the normalised target directly.
- **PC**: bidirectional Chamfer (or QAL) on the full (B, `target_points`, 3) cloud (no masking — the whole cloud is reconstructed every step), scaled by `pc_loss_weight`.
- **(4M)**: Smooth-L1 on params, masked by `text_valid` (real leaf tokens only) **and** `mask_text` (model only loses on tokens it didn't see). Scaled by `spline_loss_weight`.

`embodied_mae_4m.py` imports `PatchEmbed`, `PointCloudEmbed`, `TransformerBlock`, `chamfer_distance`, and `get_2d_sincos_pos_embed` from `embodied_mae.py` — when changing these, expect both models to be affected.

Param normalisation (4M only)

Plant and leaf parameters are normalised to [0, 1] via fixed `_PLANT_SCALE` / `_PLANT_SHIFT` and `_LEAF_SCALE` / `_LEAF_SHIFT` arrays at the top of `embodied_mae_4m.py`. `decode_params_to_text` un-normalises them back into the human-readable strings shown in visualisations. If new fields are added to the spline YAMLs, both arrays and the `_plant_to_params` / `_leaf_to_params` builders must be updated.

Dataset layout

SorghumDataset expects:

```
data_root/{train,val,test}/{sample_name}/
  rgb.png           # 224-ready RGB render
  depth.png         # big-endian packed RGBA depth
  *_nc_cam.ply      # camera-frame, normals-cleaned point cloud
  *_spline.yml      # 4M only - procedural generation params
```

Sample folders are named `Sorghum_{plant}_{view}`, so `Sorghum_0_00` ... `Sorghum_0_09` are ten views of one plant. **Only those four files are read by the loaders.** The `Sorghum_{n}.obj` and `Sorghum_{n}_nc.ply` that sit alongside them are source assets, are duplicated in full into every one of a plant's ten view folders, and are never opened during training — they are ~64 % of the bytes on disk.

The PC loader uniformly samples / pads to `num_points`, then centres at the centroid and scales so the max-distance point lands on the unit sphere. RGB uses ImageNet mean/std normalisation; depth is loaded as single-channel L and only `ToTensor`'d (no normalisation at load — normalisation happens inside the loss).

The active dataset on Nova is `/work/mech-ai-scratch/alloy/shorgum_data/new_data_50K/Sorghum_15K/`, an extreme-enriched 70/15/15 split (seed 42) of 15 000 plants: **105 000 train / 22 500 val / 22 500 test** view-samples. `assignment.csv` and `features.csv` at that root record the split; the tooling to regenerate or reshuffle it is in `data_split/`.

Porting to another machine

Everything machine-specific is in three places — nothing else needs touching:

1. **data.data_root in the YAML you run.** Every config hardcodes the Nova absolute path above.
2. **The #SBATCH headers in `slurm/*.sbatch`** — `--partition`, `--account`, `--gres`, `--cpus-per-task`, `--mem`. These encode Nova's partitions (`nova`, `scavenger`) and accounts (`mech-ai`, `mech-ai-savenger`) and mean nothing elsewhere. On a non-SLURM box, ignore `slurm/` entirely and use the `torchrun` line above.
3. **The `conda env. environment.yml` rebuilds `det`;** it pins a CUDA 12.4 PyTorch, so a different GPU generation may need a different build (see the `sm_120` note under Environment).

Moving the data is the expensive part. At ~5.0 MB per sample folder × 150 000 folders the split is ~**750 GB**, but the four files training actually reads total ~1.8 MB per sample, so a filtered copy is ~**265 GB** — under 40 % of the naive transfer. Copy with an include-filter rather than syncing the tree:

```
rsync -a --info=progress2 \
  --include='*/' \
  --include='rgb.png' --include='depth.png' \
  --include='*_nc_cam.ply' --include='*_spline.yml' \
  --exclude='*' \
  /path/to/Sorghum_15K/ user@host:/dest/Sorghum_15K/
```

Also copy `assignment.csv` and `features.csv` from the split root, and any warm-start checkpoint you need (`best_model.pth` is ~1.3 GB per run).

Note the dataset loaders build a folder index on first use and cache it — the first run on a fresh copy pays a one-off scan (~24 min for the 105 k train split); later runs print `index cache hit`. The pipeline is **dataloader-bound**, **not GPU-bound**: `num_workers` (\$ \$1.6 items/s per worker) sets throughput, so give it as many CPUs as the node allows and scale `--mem` with the worker count (~2.3 GB RSS per worker plus ~10 GB per node for model and CUDA context).

Outputs

Each run writes to `<output_dir>/:` - `checkpoints/checkpoint_epoch_{N}.pth` every `save_freq` epochs - `best_model.pth` when val loss improves - `visualizations/epoch_{N}_sample_{i}_{name}.png` every `viz_freq` epochs (4-row grid for 3M, 5-row grid for 4M including text predictions); skipped automatically when the run is a reduced-modality arm - `training_history.json` (rolling) - `config.json` — snapshot of the effective args. **Check this to confirm what a run actually used**, especially `batch_size × world_size`, `active_modalities`, `max_plants` and `model_size`.

`outputs/` is gitignored apart from a small whitelist in `.gitignore`. Wandb logging is on by default (`use_wandb: true`, project `embodied-mae-sorghum`); project and run names differ between runs — check the YAML, not the script defaults.

Things that look like dead code but aren't

- `outputs_sorghum_*/` directories at the repo root are old run outputs kept for reference; the canonical output root is `./outputs/`.
- `tools/process_depth_bg.py`, `tools/process_mask.py`, `tools/validate_sorghum_data.py`, `sweeps/vis.py`, `tools/check_structure.py` are one-off data-prep / diagnostic scripts, not part of any pipeline.
- `sweeps/vis_pc_masking.py` is a standalone tool for visualising the FPS + Dirichlet masking on a single point cloud.
- `tools/visualize_sorghum_pointclouds.py` renders multi-view PC galleries from raw `.ply` files; it doesn't touch the model.
- `eval/analyze_e2.py` reads the E2 arm output dirs and builds the modality-value-add comparison.